

SOLUCIÓN TAREA SEMANA 2: DIVIDIR Y CONQUISTAR

PROFESOR: Federico Melo Barrero

1. a) Caso 1: El arreglo es $[0]$ y el rango $[0, 0]$. Evidentemente el único periodo posible es el día 1, con cambio acumulado $0 \in [0, 0]$.

Caso 2: Para $[5]$ y rango $[-3, 3]$, $5 \notin [-3, 3]$, hay 0 periodos.

Caso 3: Para $[4, -6, 3, -1]$, los 10 periodos posibles son:

1. $[4]$, con cambio acumulado 4
2. $[4, -6]$, con -2
3. $[4, -6, 3]$, con 1
4. $[4, -6, 3, -1]$, con 0
5. $[-6]$, con -6
6. $[-6, 3]$, con -3
7. $[-6, 3, -1]$, con -4
8. $[3]$, con 3
9. $[3, -1]$, con 2
10. $[-1]$, con -1

De esos, cinco están en el rango $[-2, 2]$.

Caso 4: Para $[1, -1, 1, -1]$ un análisis similar al anterior muestra que solo cuatro periodos: $[1, -1]$ (dos veces, en los días del 1 al 2 y del 3 al 4), $[-1, 1]$ y $[1, -1, 1, -1]$, tienen cambio acumulado $0 \in [0, 0]$.

- b) Se puede formular infinidad de relaciones entre este problema y los vistos en clase, que son: merge sort (clase 2), conteo de pares invertidos y máximo subarreglo (ambos clase 3).

Una primera relación evidente es que en los cuatro problemas se resuelve la mitad izquierda, la mitad derecha y luego se combinan las soluciones de alguna manera. Haber hallado una relación de ese estilo basta (por la intencional ambigüedad del enunciado de la tarea).

Sin embargo, la solución clásica del problema define una función f como $f(k) = \sum_{i=1}^k c_i$. Es decir: f computa la suma acumulada de los primeros k elementos.

Si se usa esa función como arreglo, es decir, se define un arreglo `accum` donde `accum[i]` corresponde a $f(i)$, en ese arreglo, el cambio acumulado del periodo de días desde $i + 1$ hasta j es `accum[j] - accum[i]`

Con esa transformación es simple calcular el cambio acumulado de cualquier periodo de días usando un par de índices i, j , por lo cual es sencillo relacionar la solución al problema de pares invertidos. Como vimos en la corrección del quiz 1 en la clase 3, una solución a ese problema consiste en añadir unas pocas líneas a merge sort.

2.

```
1 def contar_periodos_aux(prefijos: list[int], lo: int, hi: int) -> int:
2     if lo >= hi: return 0
3
4     mid = (hi - lo) // 2 + lo
5
6     left_count = contar_periodos_aux(prefijos, lo, mid)
7     right_count = contar_periodos_aux(prefijos, mid+1, hi)
8     merge_count = merge_and_count(prefijos, lo, mid, hi)
9
10    return left_count + right_count + merge_count
11
12
13 def merge_and_count(prefijos: list[int], lo: int, mid: int, hi: int) -> int:
14     left = prefijos[lo:mid+1]
15     right = prefijos[mid+1:hi+1]
16
17     lo_ptr = hi_ptr = 0
18     periodos_count = 0
19     for p in left:
20         while lo_ptr < len(right) and right[lo_ptr] < p + lower:
21             lo_ptr += 1
22         while hi_ptr < len(right) and right[hi_ptr] <= p + upper:
23             hi_ptr += 1
24         periodos_count += hi_ptr - lo_ptr
25
26     # En este espacio va el paso de combinar de merge sort,
27     # tal y como se implementó en la clase 2.
28
29     return periodos_count
```

Se reproduce solo la parte conceptualmente significativa de la implementación. En `merge_and_count` se omiten los parámetros `lower` y `upper`. El archivo `solucion_tarea_2.py` contiene la implementación completa.